Smart Dental Clinic System

AI Module Documentation

(Detection + Multi-Label Classification + App Integration)

This document is derived from the project scripts (YOLO + ResNet + Streamlit) and describes the full AI pipeline, data preparation steps, evaluation, and integration with the backend database.

1. Executive Summary

The AI component of the Smart Dental Clinic System provides a complete workflow for dental image analysis. It combines: (1) object detection to localize lesions on clinical images, (2) multi-label classification to predict which conditions are present, and (3) a Streamlit user interface that generates a clinical report and saves results to a backend database through REST APIs.

1.1 Main AI Outputs

- Detection output (YOLOv8): bounding boxes + class labels for each detected lesion (localization).
- Classification output (ResNet50 multi-label): probabilities per condition + binary decisions using a threshold.
- Clinical triage: automatic urgency level and suggested action based on predicted conditions.
- Export to database: patient record + AI results saved via backend API endpoints.

2. System Architecture (AI Pipeline)

At inference time, the pipeline works as follows:

1. User uploads an image in the Streamlit interface.
2. YOLOv8 model performs lesion localization (bounding boxes).
3. ResNet50 model performs multi-label diagnosis (probabilities for each label).
4. A report is generated (detected diseases, confidence scores, and triage level).
5. The system saves the patient and the diagnostic record into MongoDB via Node.js API.

2.1 Script Map (Where each part lives)

Component	Primary Script(s)	Purpose
Detection dataset preparation	aggregate_clean_and_split.py	Aggregate class folders, clean pairs, split train/val.
Detection training	train_yolov8.py	Fine-tune YOLOv8m with stable augmentation settings.
Detection evaluation	evaluate_yolo.py	Run final evaluation on the test split defined in data.yaml.
Classification label building	build_labels_and_splits.py	Create CSV labels and train/val/test splits.
Classification training	train_multilabel_resnet.py	Train ResNet50 for multi-label classification.

Classification evaluation	evaluate_model.py	Generate test metrics and reports.
Application integration	app.py	Streamlit UI: inference, triage, and saving to backend.

3. Data Preparation and Processing

A high-quality dataset is the foundation for reliable medical AI. This project uses two datasets/pipelines: one for detection (YOLO format) and one for classification (CSV labels).

3.1 Detection Dataset (YOLO Format)

The detection dataset is organized by disease folders and then aggregated into a single YOLO-ready structure. Each image has a corresponding label file in YOLO text format: one line per bounding box as "class_id x_center y_center width height" normalized to the image size.

3.1.1 Aggregation

The script aggregate_clean_and_split.py copies images and label files from each disease folder into temporary directories before cleaning and splitting.

3.1.2 Cleaning and Synchronization

The cleaning phase enforces data integrity by keeping only valid image/label pairs:

- The label file must exist for each image.
- The label file must be non-empty (file size > 0).
- Unmatched images or empty label files are removed to prevent training noise and runtime errors.

3.1.3 Train/Validation Split

After collecting valid pairs, the script shuffles the dataset and splits it into Train and Validation sets using a fixed ratio (VAL_RATIO = 0.20). Files are then moved into the standard YOLO directory layout:

- images/train, labels/train
- images/val, labels/val

3.2 Detection Dataset Configuration (data.yaml)

YOLO training and evaluation are controlled by data.yaml. It defines the dataset root path and the relative locations of train/val/test images, as well as the class mapping (nc and names).

3.2.1 Detection Class Mapping

Class ID	Class Name
0	Data_caries
1	Mouth_Ulcer
2	Tooth_Discoloration
3	Gingivitis

3.3 Classification Dataset (CSV Multi-Label Targets)

For classification, we build a CSV file where each row represents an image and its label vector (multi-hot encoding). The script build_labels_and_splits.py scans disease folders and generates train/val/test splits.

3.3.1 Label Space (7 labels)

Index	Label
0	Healthy
1	Data caries
2	Gingivitis
3	Mouth Ulcer
4	hypodontia
5	Tooth Discoloration
6	Calculus

3.3.2 Split Strategy

The CSV is split using scikit-learn train_test_split as follows:

- Test set: 15% of the full dataset.
- Validation set: 15% of the full dataset (derived from the remaining 85% using a ratio ~0.17647).
- Train set: the remaining ~70%.

3.3.3 Important Implementation Note (CSV Delimiter)

Both train_multilabel_resnet.py and evaluate_model.py read CSV files using a semicolon separator (sep=';'). Therefore, the generated CSV files must be semicolon-separated OR the

scripts must be updated to read the default comma separator. This detail is critical to avoid silent parsing errors or misread columns.

3.4 Additional Data Processing Steps (Performed outside the shared scripts)

In addition to the processing implemented in the scripts, the following steps are commonly performed during medical dataset preparation. These steps may be executed manually or via separate utility scripts and are recommended to be documented in the thesis/defense.

- Manual annotation quality assurance (QA): verifying box accuracy and correct class IDs across all label files.
- Duplicate removal (de-duplication): removing repeated images to reduce data leakage and overfitting.
- Image standardization: ensuring consistent color mode (RGB) and filtering corrupted images that fail to load.
- Class-ID remapping: confirming that YOLO class IDs in .txt files match the order defined in data.yaml.
- Dataset sanity checks: counting images vs labels, verifying no missing pairs, and validating that all boxes are within [0, 1] normalized range.
- Balancing considerations: monitoring class distribution and adjusting sampling/augmentation strategy if classes are heavily imbalanced.
- Split leakage prevention: ensuring near-duplicate or same-patient images do not appear in both train and validation splits.

4. Detection Module (YOLOv8)

4.1 Objective

The detection module identifies and localizes oral/dental conditions by outputting bounding boxes and class labels. This supports clinicians by highlighting suspicious regions on the image and providing interpretability (where the model is focusing).

4.2 Training Configuration (train_yolov8.py)

Key training settings used in the stable fine-tuning configuration:

- Model: YOLOv8m (yolov8m.pt) as a balance between accuracy and stability.
- Epochs: 150.
- Image size: 640.
- Batch size: 8 (tuned for GPU memory).
- Reduced augmentations to avoid instability (mosaic, rotation, translation, HSV, scale).

4.3 Evaluation (evaluate_yolo.py)

Final evaluation is performed using Ultralytics validation on the split named 'test' as defined in data.yaml. Metrics and curves (Precision, Recall, mAP50, mAP50-95) are saved to the evaluation output directory.

4.4 Detection Metrics (What they mean)

Detection uses standard object detection metrics:

- Precision: proportion of predicted boxes that are correct (low false positives).
- Recall: proportion of ground-truth objects that were detected (low false negatives).
- mAP@0.50 (mAP50): mean Average Precision at IoU threshold 0.50 (easier criterion).
- mAP@0.50:0.95 (mAP50-95): mean AP averaged across IoU thresholds 0.50 to 0.95 (stricter, more informative).

5. Classification Module (ResNet50 Multi-Label)

5.1 Objective

The classification module predicts which conditions are present in the image using multi-label classification. Unlike single-label classification (one class only), multi-label allows multiple conditions to be present simultaneously.

5.2 Model Architecture

A pretrained ResNet50 backbone is used. The final fully-connected layer is replaced with a 7-unit output head followed by a Sigmoid activation. Sigmoid is used (instead of Softmax) because each label is treated as an independent probability.

5.3 Training Setup (train_multilabel_resnet.py)

Key training settings:

- Input size: 320×320; transforms include Resize + ToTensor.
- Loss: Binary Cross Entropy (BCELoss) with Sigmoid outputs.
- Optimizer: Adam, learning rate = 1e-4.
- Selection criterion: best model is saved based on highest validation micro-F1.

5.4 Evaluation Setup (evaluate_model.py)

The evaluation script loads the saved best_model.pth, runs inference on test.csv, and computes a classification_report as well as TP/FP/FN/TN per class. Predictions are binarized using a threshold of 0.5.

5.5 Classification Metrics (Precision, Recall, F1)

For each class, we report Precision, Recall, and F1-score:

- Precision = TP / (TP + FP). High precision means fewer false alarms.
- Recall = TP / (TP + FN). High recall means fewer missed cases (important in medical screening).
- F1 = harmonic mean of precision and recall. Useful when you need a single balanced metric.

Micro-averaging (micro-F1, micro-precision, micro-recall) aggregates decisions over all labels and samples, and is often preferred when there is class imbalance.

5.6 Known Limitation and Recommended Upgrade

The current label building method assigns labels primarily based on the folder name (one label per folder). If an image truly contains multiple diseases, folder-based labeling can under-represent multi-label reality. A recommended upgrade is to generate classification labels directly from the YOLO label files by marking a label as 1 if that class_id appears in any bounding box for the image.

6. Clinical Logic (Triage)

After classification, the application maps predicted diseases to a triage priority level to support clinical workflow.

- High Priority: urgent conditions requiring quick attention.
- Medium Priority: schedule soon.
- Low Priority: routine follow-up.

This rule-based triage is implemented in the app as a mapping from disease names to (priority, action).

7. Streamlit Application (Inference + Reporting)

7.1 User Workflow

6. Enter patient information in the sidebar.
7. Upload a dental image (JPG/PNG).
8. View the image and AI outputs (YOLO boxes + classification report).
9. Press 'Save to Database' to store patient + record.

7.2 Model Loading and Caching

Models are loaded once using Streamlit resource caching to reduce repeated initialization time. YOLO weights are loaded from the trained best.pt file, and ResNet weights are loaded from best_model.pth.

7.3 Report Generation

The application converts predicted probabilities to human-readable results and shows confidence percentages per predicted disease. If no disease exceeds the threshold, the system reports 'Healthy'.

8. Backend Integration (Saving Results to Database)

Saving is implemented as a two-step transaction-like flow to preserve relational linking between Patients and Records.

8.1 Two-Step Save

10. POST /api/patients: create the patient entry and return the generated patient _id.
11. POST /api/records: create a diagnostic record containing patientId, aiResult, imagePath, and doctorNotes.

8.2 Payload Overview

Patient payload example fields: name, age, gender, phone, history/notes.

Record payload example fields: patientId, imagePath (placeholder or stored path), aiResult (JSON), doctorNotes.

9. Work Breakdown (3 AI + 3 Backend)

The project can be presented efficiently by distributing responsibilities as follows:

9.1 AI Team (3 roles)

Role	Scope	What to present in the defense
AI-1: Detection Lead	YOLO dataset + training + evaluation	Data.yaml mapping, cleaning/splitting pipeline, YOLO training settings, detection metrics (mAP/PR curves).
AI-2: Classification Lead	CSV labels + ResNet training + evaluation	Multi-label concept, Sigmoid vs Softmax, F1/precision/recall, test report and confusion analysis.
AI-3: Integration Lead	Streamlit inference + triage + reporting	End-to-end demo: upload → inference → report → save; triage rationale and thresholds.

9.2 Backend Team (3 roles)

Role	Scope	What to present in the defense
BE-1: API Lead	Express routes + request/response handling	REST endpoints, status codes, request validation, error handling.
BE-2: Database Lead	MongoDB schema design (Patient/Record) + relations	Why Patient vs Record, one-to-many relationship, storing aiResult for audit trail.
BE-3: Integration/QA Lead	End-to-end reliability +	How Streamlit calls the API,

	deployment checklist	CORS rationale, demo runbook, future improvements (image upload, auth).

10. Reproducibility (How to Run)

10.1 Detection

1) Prepare dataset (aggregation/cleaning/split):

```
python aggregate_clean_and_split.py
```

2) Train YOLOv8:

```
python train_yolov8.py
```

3) Evaluate YOLO on test split:

```
python evaluate_yolo.py
```

10.2 Classification

1) Build CSV labels/splits:

```
python build_labels_and_splits.py
```

2) Train ResNet50 multi-label classifier:

```
python train_multilabel_resnet.py
```

3) Evaluate on test set:

```
python evaluate_model.py
```

10.3 Application and Backend

1) Start backend server (Node.js):

```
node server.js
```

2) Run Streamlit app (important):

```
streamlit run app.py
```

11. Future Work

- Introduce a dedicated, truly independent test set (not reusing validation).
- Build true multi-label classification targets from YOLO annotations (multi-hot labels).
- Tune per-class thresholds to balance recall and precision in medical screening.
- Store uploaded images on the server and save real imagePath in the database.
- Add authentication/authorization for doctors and patient privacy controls.
- Add a reporting module (PDF export) and analytics dashboard (case statistics).

Future Work

Planned enhancements to extend the system's clinical value:

- NLP Chatbot (Doctor & Patient Q&A): Integrate an NLP-based assistant that answers clinician and patient questions, explains model outputs in plain language, and provides guidance aligned with approved medical content and clinic protocols.
- Automatic Medical Report Generation: Generate structured clinical reports from detection/classification results (findings, confidence, triage level, recommended actions, and doctor notes), exportable as PDF/Word, and stored alongside the patient record in the database.

Future Work (Detailed Roadmap)

The current system focuses on vision-based dental diagnosis (object detection + multi-label classification) and persistence of results. The next phase extends the product to a clinically useful assistant that can communicate with doctors and patients, generate standardized reports, and support follow-up care.

1) NLP Chatbot (Doctor + Patient Assistant)

Goal: Provide a conversational interface that answers questions safely and consistently.

Key capabilities:

- Doctor mode: explain AI findings, highlight limitations, and reference clinical guidelines (evidence-based).
- Patient mode: provide simple explanations, home-care advice, and red-flag symptoms that require urgent care.
- Bilingual support (Arabic/English) and terminology control (medical vs. simplified language).

Recommended technical design (high level):

- RAG (Retrieval-Augmented Generation): store curated dental guidelines/FAQ in a vector database, retrieve relevant sections per question, then generate an answer grounded in retrieved text.
- Conversation memory scoped per patient visit: optionally include patient history + most recent AI outputs when generating responses.
- Safety layer: enforce disclaimers, block unsafe requests, and always recommend clinician confirmation for diagnosis/treatment decisions.
- Evaluation: create a set of medical Q&A test cases and measure factuality, completeness, and safety compliance.

2) Automated Medical Report Generation

Goal: Generate a standardized clinical report after each scan, ready for review and printing.

Report content (structured):

- Patient demographics + visit date/time.
- AI summary: detected conditions, confidence scores, and triage level.
- Localization evidence: annotated image(s) with bounding boxes and cropped findings.
- Clinical notes: doctor notes + patient history field.
- Recommendations: next steps, follow-up interval, and patient education summary.
- Audit section: model version, training date, threshold settings, and known limitations.

Implementation options:

- Template-based generation (recommended baseline): deterministic report layout ensures consistent clinical formatting.
- LLM-assisted narrative: generate a short readable paragraph from structured outputs, with strict constraints and references to avoid hallucinations.
- Export formats: PDF for printing + JSON for storage + Word for editing if required by the clinic.

3) Backend + Product Extensions

- Authentication and role-based access (doctor/admin).
- Secure image storage (local + cloud) and store URLs in the Record model.
- Dashboard for patient history (records timeline) and model performance monitoring.
- Continuous dataset growth: data versioning, annotation QA, and scheduled re-training.